Agent Guardrails

This document outlines the security architecture and guardrail pipeline for AI agents. By utilizing lightweight classifiers and strict code-based validation checks, security enforcement remains fast and reliable without delaying tool execution.

1. Guardrail Components

Content Filter (Input & Output)

- **Model:** ibm-granite/granite-guardian-hap-125m
- **Threshold:** Block when the toxicity probability is 0.5 or higher.

Notes: Preferred small English classifier for hate, abuse, and profanity (HAP). The 38M Slate variant (classification_transformer_en_slate.38m.hap) can be used as a CPU fallback if the 125M model exceeds the latency budget.

Prompt Injection & Jailbreak Detection

- **Model:** meta-llama/Llama-Prompt-Guard-2-86M
- **Scope:** User message only.

Performance: English AUC is 0.998; catches 97.5% of attacks at a 1% false-positive rate.

Schema Check

- **Type:** Code-based rule (no model).

Action: Enforces maximum length, permitted characters, and valid fields for structured queries.

Rule Checks

- **Type:** Code-based rule (no model).
- **Scope:** Executed on user messages and every tool call.

Action: Enforces length limits, tool allowlists, and argument bounds.

Hallucination Prevention

- **Model:** vectara/hallucination_evaluation_model (HHEM-2.1-Open)
- **Scope:** Final answer only, when source context has been retrieved (skip if no source text exists).

Action: Scores each sentence against retrieved sources (0 = unsupported, 1 = supported). Blocks responses scoring below 0.5.

Safety Classification & Moderation

- **Model:** hivetrace/gliner-guard-uniencoder (147M encoder for violence, sexual content, self-harm, and crime)

Enforcement: Acts as a secondary moderation vote alongside the IBM HAP model. Content is blocked if either model flags a violation.

Output Formatting

- **Type:** Code-based rule (no model).

Action: Validates responses against JSON Schema when structured JSON is required. Returns an immediate error on failure rather than retrying through the agent.

PII Redaction

- **Tool:** Microsoft Presidio with en_core_web_sm pipeline.
- **Scope:** User messages, tool results, and final answers.

Action: Detects and masks emails, phone numbers, credit cards, and national IDs (including checksum verification). The agent continues processing using the masked text.

Secret Masking

- **Type:** Code-based regular expressions (no model).

Action: Scans for API keys, access tokens, and private key blocks across user messages, tool results, and final answers, masking detected spans.

Phase 1: Input Evaluation (User Message)

- **Executed Checks:** Rules, Prompt Guard 2, HAP-125m, GLiNER Guard safety labels, Presidio PII redaction, Secret regex, and Intent classification.
- **Behavior:** Block turn if any safety score trips.

Handoff: The agent receives the masked text, classified intent, and the permitted tool allowlist.

Phase 2: Tool Calls

- **Executed Checks:** Code rules only (allowlist, argument schema, secret regex).

Performance: No AI model calls are made during tool execution to maintain sub-millisecond execution.

Phase 3: Output Evaluation (Final Answer)

- **Executed Checks:** HAP-125m, GLiNER Guard, Presidio PII redaction, and Secret regex.
- **Hallucination Check:** Evaluates grounding using HHEM if documents were retrieved; blocks score below 0.5.

Schema Check: Validates against JSON Schema if output requires structured formatting.

3. Performance Limits

- **Input Gate Budget:** ~100 ms max.
- **Output Gate Budget:** ~150 ms max.
- **Tool Checks:** Under 5 ms total.

Timeout Action: Deny turn if any model exceeds the target budget.

Custom Training Scope: Train only the intent classifier using bge-small-en-v1.5 embeddings and logistic regression. All other guardrails (HAP-125m, Prompt Guard 2, GLiNER Guard, HHEM, Presidio) are used off-the-shelf as published.

AI agents without guardrails aren't powerful, but they're risky.

9 layers every team needs before going to production...

AI agents are only as reliable as the guardrails behind them.

This is no longer optional; it's the difference between scaling safely or failing fast.

Here are the essential guardrails every AI agent needs today:

1/ Content Filtering: Input + Output -sentence transformer / classification_transformer_en_slate.38m.hap for HAP detection.https://dataplatform.cloud.ibm.com/docs/content/wsj/analyze-data/watson-nlp-block-hap.html?context=cpdaas

Stops harmful, sensitive, or non-compliant data before it enters or leaves your system.

2/ Input Validation: Query Stage–classifier (skillsfactor by nvidia)

Prevents prompt injection,
enforces schema rules
ensures structured inputs reach the agent clean.

3/ Intent Recognition *

Understands what the user actually wants, critical for correct tool routing and planning decisions.

4/ Rule-Based Checks: Pre-Processing— 2

Lightweight filters (regex, limits, constraints) that catch edge cases before reasoning even starts.

5/ Hallucination Detection: SLMs + Evaluators

Flags low-confidence or fabricated outputs before they ever reach a user.

6/ Safety Classification: Specialized Models

Classifies queries in real-time to block unsafe or restricted actions at the gate.

7/ Moderation Layers: APIs + Internal Models

Adds redundancy across input and output because one layer is never enough in production.

8/ Output + Format Validation

Ensures responses are usable (JSON, SQL, API-ready) and won't break downstream systems.

9/ Sensitive Data Detection: PII + Secrets

Prevents leakage during both retrieval and generation. Non-negotiable for any enterprise RAG pipeline.

Official OWASP ACS page- https://genai.owasp.org/resource/agent-control-standard-acs/
— introduction and overview.
ACS GitHub repository- https://genai-security-project.github.io/agent-control-standard/
— specification, schemas, and reference implementation.
ACS documentation and specification-
— detailed technical documentation.

Intent Classification

- **Model:** BAAI/bge-small-en-v1.5 + logistic regression trained on custom queries.

Action: Label a few dozen real queries per intent. Restrict the agent to read-only tools if the top intent confidence score is below 0.6.

Sales Assistant

Thumbs-down

- A seller rates an answer. The app saves the rating on that chat's Langfuse trace and posts it in the team Slack channel.
- A thumbs-up stops there. A thumbs-down continues after the page has already said the rating was saved.
- The app starts a GitLab job whose prompt is "analyze this one trace." Tests and deploys do not run.
- The job starts Claude with the `ci-agent` plugin. The prompt `/ci-agent:self-improving` makes Claude open `SKILL.md`.
- The prompt contains a trace id, so Claude follows the explicit section of that recipe.
- Helpers fetch three slices of that chat: the tool calls, the question and answer, and the thumbs-down score.
- Another helper opens `explicit-feedback.md` and `shared-registry.md` and returns one decision.
- No ticket is opened for an access problem, a known limit, an out-of-scope question, a misunderstanding, or a document that is simply outdated. The seller gets a short Slack explanation.
- A real product failure opens a Jira ticket labeled `sa-feedback`, or adds a note if that bug already exists.
- `post_slack_threads.py` replies under the original Slack message with the question, the diagnosis, and the ticket link, and can DM the seller that the feedback was received.
- When that ticket is later marked Done, a second job opens `ci-jira` and runs `close_feedback_loop.py`.
- That script DMs the seller that the fix is live, or that the requested feature is available, and marks the ticket so the message is not sent again.

Scheduled scan

- Nobody clicked a rating. The same skill starts with no trace id, so it follows the implicit section.
- It reads `implicit-triage.md` and scans recent chats for errors, very slow runs, and people who kept starting new chats.
- Only the suspicious chats are read in full, using `implicit-signals.md`.
- Chats with the same cause become one finding. There is no seller thread to answer.